TrustSQL: A Reliability-Oriented LLM-based Text-to-SQL System

Ke Lu UNI: kl3753

COMS 6156 — Topics in Software Engineering, Columbia University

May 2026

I. Synopsis

This paper mainly studies the Text-to-SQL technology that can convert natural language questions into executable SQL queries. With the rapid development of large language models (LLMs) nowadays, LLMs are widely used in various technologies, and LLMs themselves have also made significant progress in this process. However, the practical application of LLMs in the Text-to-SQL field is still not reliable enough. For instance, the model may fabricate column names, wrongly select tables, construct incorrect JOIN paths, and have poor generalization ability when the wording of the question changes. As ToolSQL [9] pointed out, many multi-stage methods rely on the limited feedback provided by the database management system for error correction, and there is insufficient exploration of stronger verification and correction mechanisms. RBDQ [6] also clearly pointed out that even if the execution result set is consistent with the standard answer, queries with different semantics may accidentally produce the same result, causing a false sense of security in evaluation. These problems are particularly prominent in the real-world benchmark BIRD, which contains noisy multi-table databases and requires domain knowledge [2, 11].

This project presents TrustSQL, a three-component reliability pipeline aimed at enhancing the accuracy and robustness of LLM-based Text-to-SQL systems. Unlike most studies reviewed in the literature, which typically focus on only one or two of the three reliability mechanisms [1], this project integrates Schema Awareness, Knowledge & Reasoning, and Guardrails & Verification into a single system pipeline and quantifies the independent contribution of each mechanism through controlled ablation experiments.

Three reliability mechanisms are integrated into the pipeline:

- **Schema Awareness:** Before generating SQL, the complete database schema is passed to the LLM, which filters out irrelevant tables and columns and returns a simplified schema related to the question. This prevents the generator from being disturbed by irrelevant fields and reduces the hallucination of non-existent column names. This approach is consistent with the multi-stage schema linking method of PSM-SQL [4] in terms of goal, but this project uses LLM-driven prompt filtering instead of embedding retrieval. If the LLM returns an incorrect format, the system automatically reverts to the complete schema.

- **Chain-of-Thought Reasoning:** Instead of allowing the LLM to directly generate SQL, it is first guided to output the intermediate reasoning process: Does the query require JOIN? Does it need aggregation or sorting? Which tables are involved? This structural reasoning is passed as context to the SQL generator to guide the generation direction. As demonstrated by SQLify Me [1] and STaR-SQL [5], chain-of-thought reasoning not only improves the final generation accuracy but also makes the decision-making process partially visible, thus having certain interpretability.

- **Verification & Self-Correction:** After the SQL is generated, the pipeline performs three checks: SQLite syntax verification, column name existence check, and JOIN path rationality check. The key design is that only syntax errors trigger the LLM's self-correction, while column name checks and JOIN checks are downgraded to warnings. This design draws inspiration from the tool layer verification

approach of ToolSQL [9], and at the same time, a conservative design is made to address false positives. The validator is designed to prefer false negatives over false positives — it is safer to miss a potential error than to incorrectly modify a correct SQL query.

TrustSQL targets two primary user communities:

- **Non-technical business users within enterprises** (e.g., data analysts, product managers): They need to query databases but cannot review SQL. The CoT reasoning process of TrustSQL enables them to understand how the system reaches conclusions, thereby truly trusting the query results.
- **Text-to-SQL researchers and developers:** TrustSQL provides an open-source three-stage reliability pipeline implementation. Ablation experiments quantify the contribution of each mechanism, which can be directly used as a baseline for reliability research.

I evaluated TrustSQL on the first 100 data points from the BIRD development set, using two LLMs: GPT-4o-mini and Llama-3.1-8b. The third model, Gemini 2.0 Flash, was excluded from the original plan because Google adjusted its API policy in April 2026, reducing the free quota to zero. The relevant code has been implemented. The evaluation metric was execution accuracy (EX) [7].

The complete pipeline increased the EX of GPT-4o-mini from 24% to 26%, and that of Llama-3.1-8b from 17% to 19%. The ablation experiment (n=20) showed that Schema Awareness contributed the most independently, achieving a $+30\%$ improvement, while CoT contributed an additional $+20\%$. Verification alone did not increase the EX but did not cause any side effects. The robustness test showed that problem rewriting and synonym substitution both led to a 10% decrease in accuracy, which was consistent with the perturbation sensitivity observed by ToolSQL [9] on SpiderMismatch.

II. System Design

2.1 Overall Architecture

TrustSQL adopts a modular pipeline design, with each component having clear responsibilities and being able to be enabled or disabled independently. The complete process is shown in Figure 1.

2.2 Component One: Schema Awareness

2.2.1 Motivation

The schema of the BIRD database typically consists of over a dozen tables and hundreds of columns. As discussed in the literature review, directly injecting the complete schema into the prompt would introduce a large amount of irrelevant information to the current problem, increasing the risk of the model selecting the wrong columns [4]. PSM-SQL [4] demonstrated the importance of schema awareness in ensuring the stability of the generated output by gradually filtering candidate tables and columns through multi-granularity schema linking.

2.2.2 Implementation

The complete schema and user question are sent to the LLM, which is requested to return the filtered schema in JSON format, retaining only the relevant tables and columns. The prompt strategy follows the principle that err on the side of retaining, ensuring that Primary Key and Foreign Key are retained, and considering whether columns that are not necessarily related should also be retained. If the returned format is not valid, the system will revert to the complete schema.

2.2.3 Experimental Notes

The ablation experiment shows that this is the single component with the greatest contribution among the three mechanisms, with an accuracy improvement of $+30\%$ EX. The BIRD database contains a large number

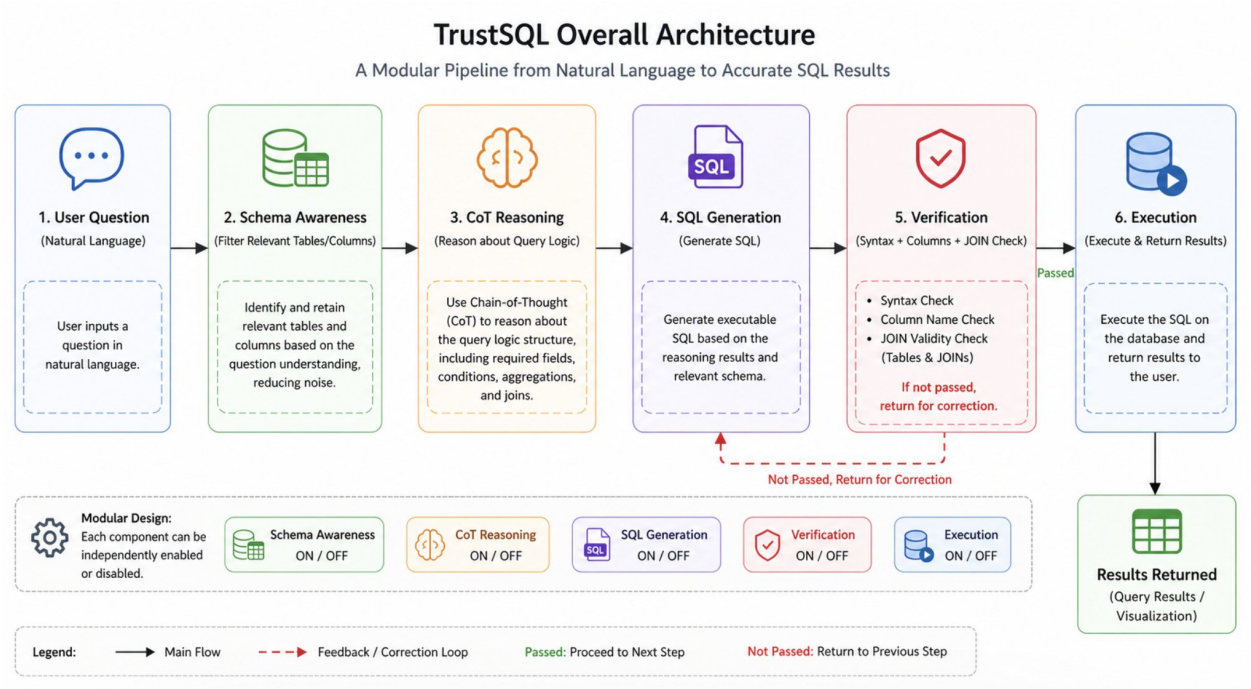

Figure 1: TrustSQL Overall Architecture. Each component can be independently enabled or disabled, supporting ablation study configurations.

of column names with spaces and special characters, such as `Free Meal Count (K-12)`. LLMs often simplify these column names when dealing with a complete schema, resulting in execution failures. After schema filtering narrows the problem to relevant columns, the model can more easily correctly reference these column names.

2.3 Component Two: Chain-of-Thought Reasoning

2.3.1 Motivation

As pointed out in Knowledge-to-SQL [10], many Text-to-SQL errors stem from misunderstandings of the problem semantics rather than structural issues, a challenge observed across domains including clinical NLP [3]. When directly asking the LLM to generate SQL, logical reasoning and column name guessing interfere with each other. SQLify Me [1] demonstrated that incorporating intermediate reasoning steps can make the decision-making process partially visible; STaR-SQL [5] further indicated that by generating reasoning samples automatically and re-training, the reasoning can be made more stable. This project achieves a similar reasoning-guided effect through prompt engineering without the need for re-training.

2.3.2 Implementation

Before generating the SQL, six structural questions are posed to the LLM:

1. What type of values need to be returned?
2. Which tables are involved?
3. Is a JOIN necessary?
4. Are there any filtering conditions?
5. Is aggregation required?
6. Is sorting or limiting the number of rows needed?

The CoT only reasons about the logical structure and does not involve specific column names. This avoids

incorrect column name guesses from contaminating the SQL generator.

2.3.3 Experimental Notes: Design Intent vs. Actual Limitations

Ideally, the CoT reasoning results should be presented to the users for confirmation. RBDQ [6] suggests that enabling business users to locate problems along the steps is the key to enhancing traceability and auditability. However, the BIRD automatic evaluation framework cannot verify the reasoning process. The dataset only contains information such as the database, related questions, and the target SQL. But this cannot answer whether the reasoning in the CoT process is correct. If an experiment to achieve this functionality is desired, a large amount of manual review is required. Manually actively determining whether the reasoning process is correct is a task that is too large for a single person to accomplish within a short period of time. Therefore, the evaluation version only uses structural reasoning for generating the context.

2.4 Component Three: Verification & Self-Correction

2.4.1 Motivation

ToolSQL [9] achieved a stronger verification capability than execution-guided decoding by introducing an external tool layer (Database Retriever and Error Detector). It provided targeted feedback in aspects such as field existence, JOIN rationality, and condition value matching. This project adopts this approach but employs a more conservative intervention strategy.

2.4.2 Implementation

The three-layer checking mechanism works as follows:

Layer 1: Grammar Check

Call SQLite's `EXPLAIN QUERY PLAN`. If a grammar error is reported, trigger LLM's self-correction. Send the original SQL, schema, and error information to LLM, requesting it to return the corrected SQL. Only adopt it if it passes the verification again.

Layer 2 - Column Name Check

Verify whether the column names referenced in the SQL exist in the filtered schema. Downgrade to a warning and do not trigger automatic correction.

Layer 3 JOIN Path Check

Check if there are Foreign Key associations between the tables involved in the JOIN operation. Downgrade to a warning and do not trigger automatic correction.

2.4.3 Experimental Notes

The initial design triggered self-correction for all three layers, resulting in the "Verification only" configuration showing a decrease in EX rather than an increase. The reason is that after schema filtering, valid column names might not be present in the filtered schema, causing incorrect judgments that triggered rewriting and corrupted correct SQL. This confirms the judgment in the literature review: Guardrails are necessary but not sufficient security layers. They can significantly reduce avoidable low-level errors, but cannot replace semantic-level modeling [1]. After the fix, the principle is: The validator is designed to prefer false negatives over false positives. It is safer to miss a potential error than to incorrectly modify a correct SQL query.

2.5 Technical Implementation

- **Language:** Python 3.10
- **LLM Interface:** Accessed through the unified abstract base class `llms/base.py`, supporting OpenAI SDK (GPT-4o-mini) and Groq SDK (Llama-3.1-8b)
- **Database Execution:** All SQL is sandboxed and executed on a local SQLite copy
- **Rate Limiting:** The free version of Groq has a limit of 6,000 TPM; implemented exponential backoff retry (1s $\rightarrow$ 2s $\rightarrow$ 4s $\rightarrow$ 8s $\rightarrow$ 16s)
- **Interactive Demo:** `main.py` is directly run to showcase the five-step processing flow

III. Research Questions

3.1 RQ1: Can the Three-Component Pipeline Improve Execution Accuracy?

3.1.1 Experimental Setup

The first 100 items from the BIRD development set, comparing Baseline with Full Pipeline across two LLMs, using metric EX [7].

3.1.2 Results

Table 1 and Figure 2 present the overall results.

Table 1: Overall Execution Accuracy: Baseline vs. Full Pipeline (n=100)

Model	Baseline EX	Full Pipeline EX	Improvement
GPT-4o-mini	24%	26%	+2%
Llama-3.1-8b	17%	19%	+2%

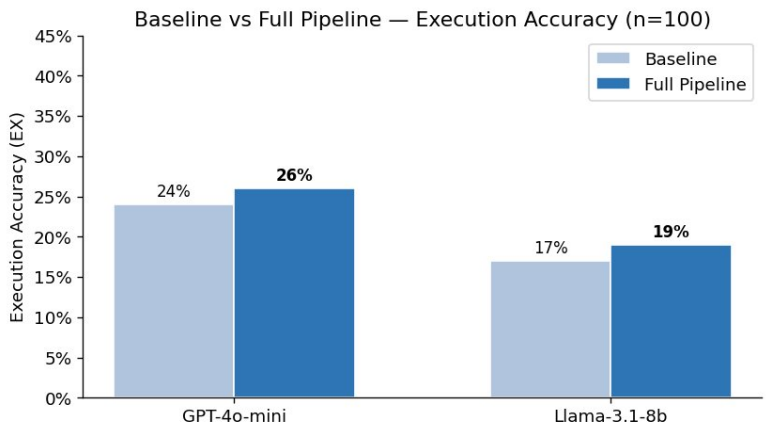

Figure 2: Baseline vs. Full Pipeline — Execution Accuracy (n=100).

3.1.3 Decomposition by Difficulty

Figure 3 shows accuracy broken down by difficulty level.

- **Simple:** GPT-4o-mini +5.1%; Llama-3.1-8b −3.4%
- **Moderate:** GPT-4o-mini +2.9%; Llama-3.1-8b +11.4%
- **Challenging:** Both models 0% (only 6 cases — not statistically significant)

3.1.4 Answer to RQ1

The pipeline is effective at both Simple and Moderate difficulty levels. The overall +2% result is negatively affected by the scarcity of Challenging samples (only 6 items). The improvements at Simple and Moderate difficulty levels are more representative.

3.2 RQ2: How Does Each Mechanism Contribute?

3.2.1 Experimental Setup

Ablation experiment on the first 20 items, five configurations, GPT-4o-mini.

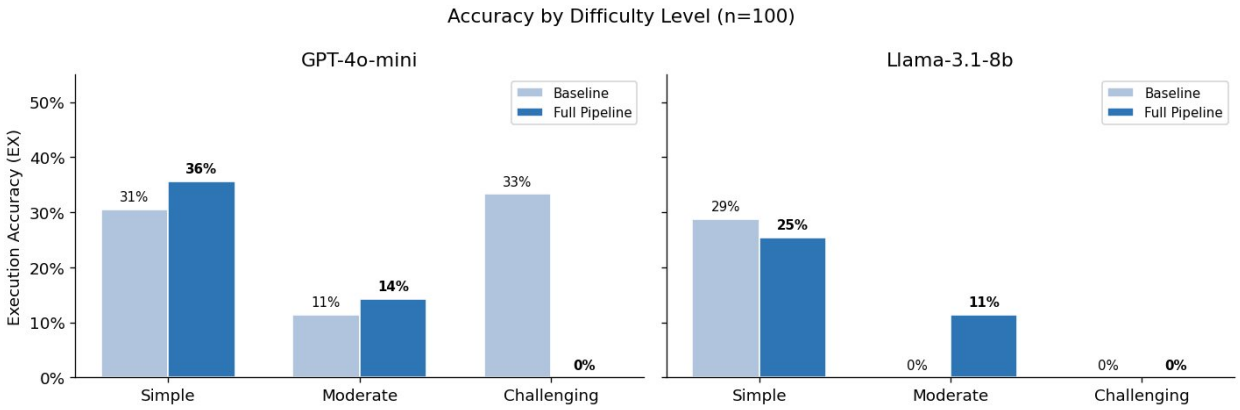

Figure 3: Accuracy by Difficulty Level (n=100).

3.2.2 Results

Table 2 and Figure 4 present the ablation results.

Table 2: Ablation Study — Each Mechanism Individually vs. Combined (n=20, GPT-4o-mini)

Configuration	**EX**	**vs. Baseline**
Baseline	20%	—
Schema only	50%	+30%
CoT only	40%	+20%
Verification only	20%	0%
Full Pipeline (all three)	45%	+25%

3.2.3 Experimental Notes

Schema Awareness made the greatest contribution (+30%). CoT had a significant auxiliary contribution (+20%). Verification alone showed no improvement in EX but caused no negative impact. The Full Pipeline (45%) is slightly lower than Schema only (50%), indicating a slight interference among the three mechanisms when combined. This is consistent with the judgment in the literature review that current research still lacks a cross-mechanism system integration framework [1].

The ablation experiment was not an academic design planned from the beginning — it emerged naturally during the debugging process. When the first round of experiments yielded poor results, I tested each of the three mechanisms one by one to locate the source of the problem. Only later did I realize this was precisely the standard academic ablation analysis method.

3.2.4 Answer to RQ2

Schema Awareness is the core mechanism. CoT provides significant auxiliary improvement. The value of Verification lies in preventing syntactic errors rather than directly enhancing EX.

3.3 RQ3: Does the Pipeline Remain Robust Under Question Perturbation?

3.3.1 Experimental Setup

The first 20 cases, Full Pipeline, GPT-4o-mini, with two semantically equivalent perturbation types.

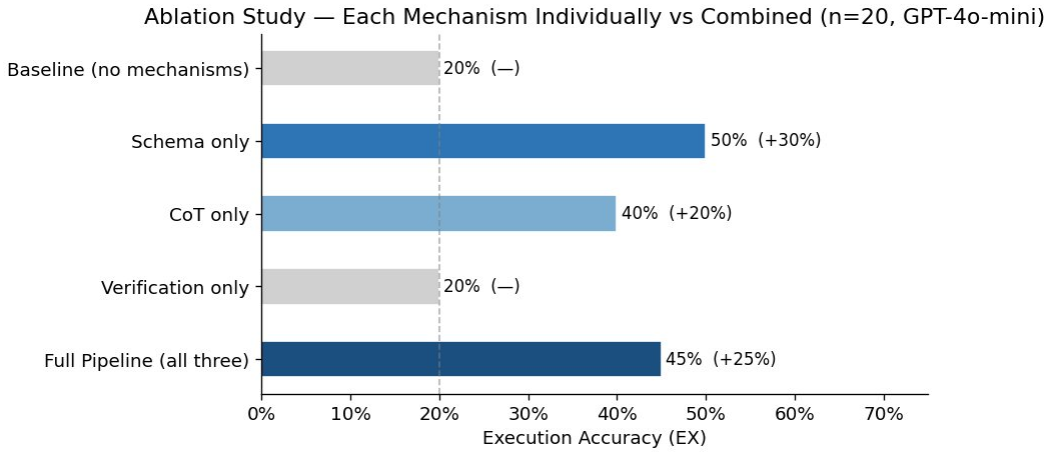

Figure 4: Ablation Study — Each Mechanism Individually vs. Combined (n=20, GPT-4o-mini).

3.3.2 Results

Table 3 and Figure 5 present the robustness results.

Table 3: Robustness — Full Pipeline under Question Perturbation (n=20, GPT-4o-mini)

Perturbation	**EX**	**Drop**
Original	40%	—
Paraphrase	30%	-10%
Synonym Replace	30%	-10%

3.3.3 Experimental Notes

Both perturbations led to a consistent 10-percentage-point decrease in accuracy. The fundamental reason lies in Schema Awareness's reliance on keyword matching — when the wording changes, keyword matching may fail and relevant columns may be filtered out. This is consistent with the phenomenon observed by ToolSQL [9] on SpiderMismatch, where many methods experienced significant performance decline under question perturbation. Future improvement directions include semantic embedding-based schema linking (referencing the approach of PSM-SQL [4]) or paraphrase augmentation [8].

3.3.4 Answer to RQ3

The pipeline is somewhat sensitive to semantically equivalent wording variations (-10%), which is an inherent limitation of keyword-based schema linking.

3.4 Supplementary Analysis

3.4.1 Error Type Analysis

Figure 6 shows the error type distribution for the Full Pipeline on 100 data points.

Wrong Column and Wrong Join are the two most common error types in both models, consistent with the error distributions reported in Knowledge-to-SQL [10] and ToolSQL [9], indicating that column name identification and multi-table association remain the core difficulties of Text-to-SQL. The proportion of Wrong Column and Wrong Join in Llama-3.1-8b is significantly higher than in GPT-4o-mini, suggesting that weaker models have a greater gap in schema understanding.

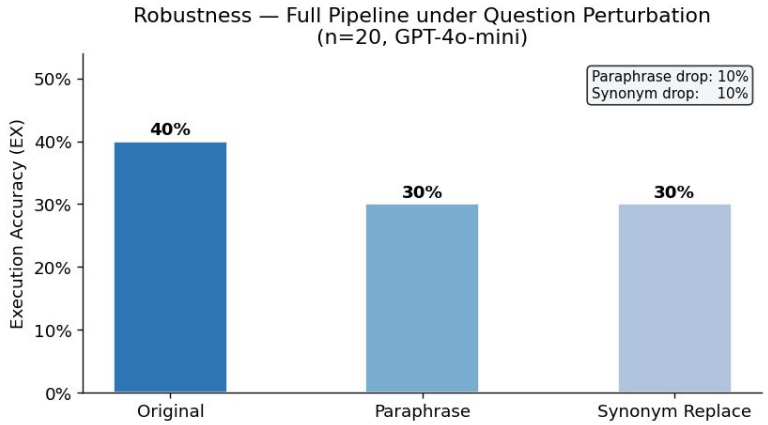

Figure 5: Robustness — Full Pipeline under Question Perturbation (n=20, GPT-4o-mini).

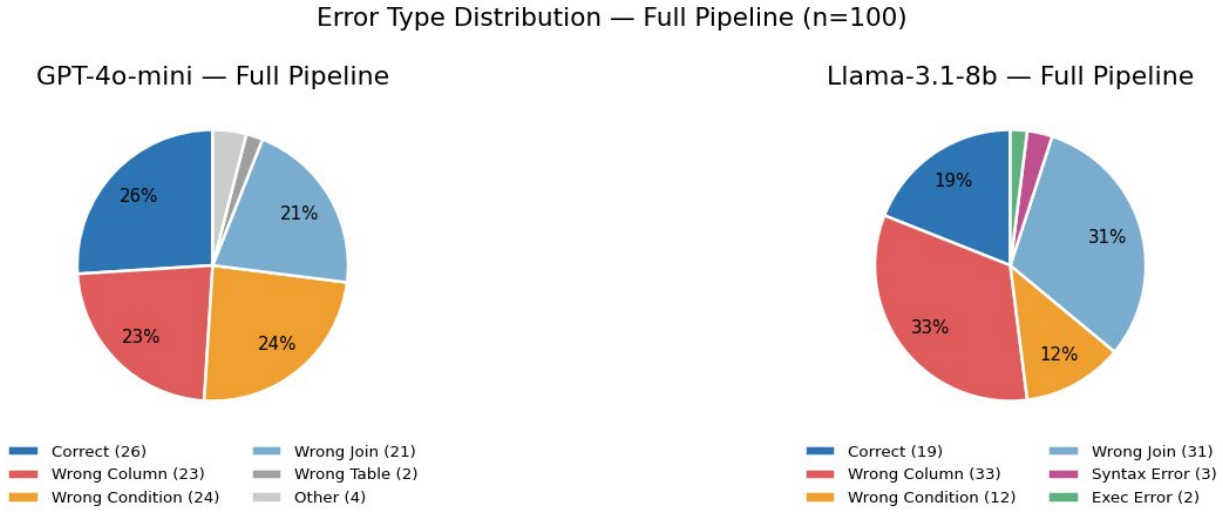

Figure 6: Error Type Distribution — Full Pipeline (n=100).

3.4.2 Efficiency Analysis

Figure 7 shows the average latency comparison (n=10, GPT-4o-mini):

- Baseline: 1.85 seconds per query
- Full Pipeline: 5.67 seconds per query
- Overhead factor: $3.1\times$

Three additional LLM calls — Schema filtering, CoT reasoning, and occasional self-correction — are the main reasons for the latency increase. For offline scenarios prioritizing accuracy, this trade-off is reasonable.

IV. Deliverables

All code, experimental scripts, and documentation are hosted at: https://github.com/LOOK4869/TrustSQL

The repository includes:

- `pipeline/` — Three core mechanisms (`schema_awareness.py`, `reasoning.py`, `verification.py`)
- `llms/` — LLM encapsulation layer (`openai_llm.py`, `groq_llm.py`, `gemini_llm.py`)

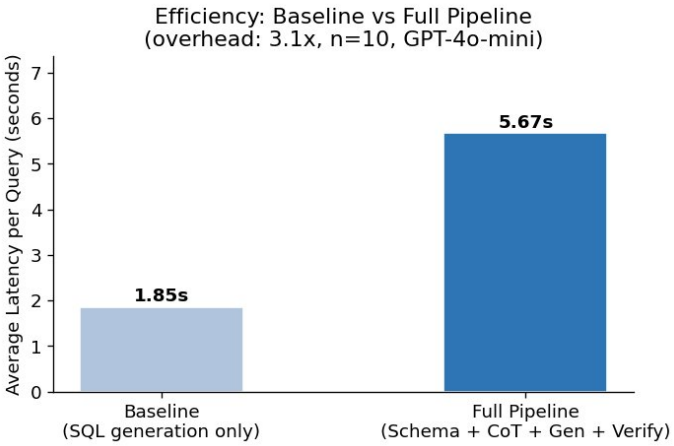

Figure 7: Efficiency: Baseline vs. Full Pipeline (overhead: 3.1×, n=10, GPT-4o-mini).

- `evaluation/` — Executor, metric calculation, robustness testing
- `experiments/` — All experiment scripts (`run_baseline.py`, `run_full_pipeline.py`, `run_ablation.py`, etc.)
- `main.py` — Interactive Demo entry point
- `notebook.ipynb` — Experiment visualizations and all result figures
- `results/` — Saved experiment JSON outputs for all reported experiments
- `milestones/` — All course milestone documents (project proposal, revised proposal, progress report, literature review, demo slides, and final report)
- `README.md` — Installation, configuration, and reproduction instructions

Interactive Demo (see Appendix Figure A1): Directly run `python main.py` to enter interactive mode. The system lists 11 available databases. After the user inputs a question, it gradually displays the outputs of the five processing stages. A demo slide deck (`milestones/TrustSQLDemo.pdf`) is also included in the repository, presenting the project overview, pipeline architecture, live run screenshots, and experiment result figures.

Reproducibility: All experiments can be reproduced by running the scripts in `experiments/` with the configuration specified in `README.md`. The BIRD development set is publicly available at https://bird-bench.github.io. API keys for OpenAI and Groq must be configured in a `.env` file as described in `README.md`. Figures and tables in this report can be reproduced by opening `notebook.ipynb` and running all cells after the experiment scripts have been executed.

AI Tools Disclosure: Claude (Anthropic) was used to assist with report writing and code debugging. All experimental designs, system architecture decisions, and reported results were produced by the author.

Collaboration Disclosure: This project was completed independently. No external collaborators outside this class were involved.

V. Self-Evaluation

5.1 Completion Status

This project has successfully achieved all the core contents promised in the project proposal and progress report, including the three-component reliability pipeline, complete experimental evaluation, two-LLM comparison, and interactive Demo mode. The full experiments included the main experiment, ablation experiment, robustness test, and efficiency analysis. The originally planned three-model comparison (including Gemini) was adjusted to a two-model comparison due to the Gemini API policy change, and an ablation experiment

was added as supplementary analysis.

5.2 Challenges Encountered

5.2.1 Gemini API Policy Changes

The project initially planned to compare three models, and the progress report clearly listed Gemini 1.5 Flash. When applying for APIs for the three models, the other two models successfully configured the APIs as expected. However, in April 2026, Google adjusted the AI Studio personal account policy in the relevant regions, reducing the free quota to zero, which made Gemini unusable for experiments. The relevant code (`llms/gemini_llm.py`) has been fully implemented, and it can be directly accessed with a quota. However, the course requires to avoid or minimize the use of expensive large models. Considering that we have decided to use the free Llama and the highly cost-effective GPT that only requires a small amount of money, it is sufficient for conducting subsequent performance comparison experiments. Therefore, we ultimately decided to abandon the expensive Gemini. This experience shows that the LLM API ecosystem changes very rapidly, and research projects need to prepare contingency plans for external dependency changes.

5.2.2 Full Pipeline Initial Version Lower than Baseline

In the first set of 100 experiments, the Full Pipeline result (13%) fell below the Baseline (18%). Investigation revealed two causes: (1) the Schema filtering prompt was overly aggressive, deleting key columns; and (2) the SQL generation prompt did not emphasize the need to use backticks for column names with spaces. After the fix, the prompt was redesigned with a conservative retention principle and explicit backtick requirements. This verified the judgment in the literature review that prompt engineering details have a decisive impact on system performance [7], and any changes require re-experimentation.

5.2.3 Unexpected Discoveries from the Ablation Experiments

The ablation experiments were not pre-planned academic designs; they were simply by-products of the debugging process. After the first round of investigation, the performance of the second round of 100 experiments remained extremely poor. Simply delving deeper into the code was too difficult to pinpoint the problem. So I thought of trying to test each of the three mechanisms separately to identify which part of the code had the issue. During this attempt, I suddenly realized that this was like an ablation experiment. It was very interesting to me, and it made me realize that the gap between good engineering intuition and good scientific methods is often closer than one might think. The choice of ablation experiments also brought me a lot of benefits. When writing the review, I always thought that more mechanisms should lead to better and better results in the end. But after conducting the ablation experiments, I discovered that the mechanisms actually restrained each other. Each of the three mechanisms tested separately showed some improvement, but when combined, the accuracy rate actually decreased. I needed to adjust the relationship among the three mechanisms based on the situation to make the three mechanisms integrate better.

5.2.4 Verification False Positive Issue

The initial three layers of checks all triggered self-correction. The ablation experiment revealed that Verification only performed worse instead of better. The fundamental reason is that there were false positives in the column name check. After Schema filtering, the valid column names might not be present in the filtered schema, which led to the LLM modifying the correct SQL. This confirms the analysis of the limitations of Guardrails in the literature review [1], which is necessary but not an adequate security layer. It cannot overly intervene at the semantic level. We need this mechanism as an insurance mechanism rather than a mechanism for improving accuracy. The solution is to downgrade the column name and JOIN checks to warnings.

5.2.5 Limitations of CoT Evaluation

The ideal design of CoT is as a user confirmation step. According to RBDQ [6], enabling business users to locate problems along the steps is the key to enhancing traceability, which is also a highlight of the Demo presentation. However, the automatic evaluation of BIRD cannot verify the reasoning steps, and the workload of manual review is too large. To achieve this mechanism perfectly requires a lot of time and manpower. Currently, as a course experiment assignment, it may not be able to be completed. Therefore, the evaluation version weakens the expression ability of CoT. Its ablation contribution +20% may be lower than the theoretical limit. This is a compromise choice, but the current version has also achieved most of the functions planned in this mechanism.

5.3 What Was Learned

From a technical perspective, I gained a deep understanding of the main failure modes of Text-to-SQL. "Wrong column" is far more common than "wrong table", which is consistent with [9, 10]. I mastered the iterative method of prompt engineering and learned how to design controlled ablation experiments to quantify the contributions of individual components.

From the experimental process, I gained a lot of experimental experience beyond Text to SQL. The first thing I learned is that the number of mechanisms is not necessarily the more the better. During the literature review stage, I chose to combine three mechanisms together, with the initial assumption that combining more mechanisms would monotonically improve accuracy. At this stage, the instructor asked me to add efficiency analysis and compare average latency. When I saw the final average latency data in the experiment, I suddenly realized that more mechanisms meant that more time would be consumed to obtain the answer, which I had not considered before. The instructor's suggestion to include efficiency analysis proved valuable, as the latency data revealed an overhead of $3.1\times$ that I had not initially anticipated, which is a gap that would be significant in production environments.

Secondly, during the process of revising the project proposal, the instructor also reminded me that a very large and excessive dataset might be beyond my handling capacity. I might have lacked relevant experience before and naively prepared to use a large number of real datasets. However, in the actual experiment, it was simply impossible to achieve this. The workload was too huge. The instructor's reminder also helped me avoid many detours and better understand my own ability range.

Then there was the ablation experiment during the experiment process. This was not part of the plan, but it played the most important role in my experiment. Because the experimental results were very poor, I thought of testing the mechanisms separately to find the problem. Unexpectedly, by accident, I started the ablation experiment in a professional sense. It was this data that made me change my mind. I originally thought that the experimental results were bad, and I would test each one separately, expecting that none of them would be high. I would then choose the worst or obviously abnormal part for modification. But the result was unexpected. When the three mechanisms ran separately, they all had very high accuracy rates, with a significant improvement. This means that what I really need to improve is not a single mechanism, but the integration of the three mechanisms. I thought that the focus of my system was to propose the innovative idea of the three-mechanism pipeline. At this moment, I realized that what my system really needs to overcome is how to better integrate the three mechanisms to achieve a better effect than the operation of any single mechanism.

The final outcome of this research largely met the expectations outlined in the literature review and the project report. However, the actual result was much less perfect than I had imagined. This project has taught me that a person's ability in a single class is limited. I had overestimated my own capabilities before. Moreover, when actually conducting the experiment, it was completely different from what I had envisioned when writing the project report. Many things do not work in the same way as described in the project report - one mechanism combined with another does not always lead to better improvement. Only when it is run on

the computer do you realize that the difficulties are much greater than I had imagined. This might be the case where one cannot envision things they have never seen before. I will learn and practice more knowledge rather than just relying on literature. If I had tried to implement the relevant code from others' papers when reading the literature, I would have definitely discovered these difficulties earlier.

In conclusion, this research has taught me that I need to undertake more practical tasks and experiments in order to become more familiar with the knowledge presented in the literature. Then, from the perspective of research methods, this project has enabled me to deeply understand the core viewpoints in the literature review. Text-to-SQL is a system engineering issue, rather than just a language generation problem [1]. The interdependence and interference among the three mechanisms indicate that system-level integration is more complex and has greater research value than the optimization of a single mechanism.

References

[1] A. Y. Mohamed, M. H. Zain-elabdeen, S. A. Sayed, S. S. Abdelwahab, Z. E. Mohamed, Z. M. Ali, Y. A. Kawashti, and H. Hindy, "SQLify Me: An Automated Text-To-SQL Query Generator," in *Proc. 2024 34th Int. Conf. Computer Theory and Applications (ICCTA)*, 2024, pp. 297–302. doi: 10.1109/ICCTA64612.2024.10974783

[2] A. Kanburoğlu and F. B. Tek, "Text-to-SQL: A methodical review of challenges and models," *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 32, no. 3, pp. 403–419, 2024. doi: 10.55730/1300-0632.4077

[3] M. Jabir, K. Kanakarajan, and M. Sankarasubbu, "Saama Technologies at EHRSQL 2024: SQL Generation through Classification Answer Selector by LLM," in *Proc. 6th Clinical Natural Language Processing Workshop*, Mexico City, Mexico, Jun. 2024, pp. 655–671. doi: 10.18653/v1/2024.clinicalnlp-1.63

[4] Z. Yang et al., "PSM-SQL: Progressive Schema Learning with Multi-granularity Semantics for Text-to-SQL," *arXiv preprint arXiv:2502.05237*, 2025. doi: 10.48550/arXiv.2502.05237

[5] M. He, Y. Shen, W. Zhang, Q. Peng, J. Wang, and W. Lu, "STaR-SQL: Self-Taught Reasoner for Text-to-SQL," in *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria, Jul. 2025, pp. 24365–24375. doi: 10.18653/v1/2025.acl-long.1187

[6] F. Bi, D. Cao, Z. Wang, Y. Chen, F. Zhao, T. Hu, Z. Li, Y. Zhang, and W. Wang, "RBDQ: A Reliable LLM-based Text-to-SQL System for Business Data Queries," in *Companion Proc. ACM Web Conference 2025 (WWW '25)*, New York, NY, USA, 2025, pp. 95–103. doi: 10.1145/3701716.3715257

[7] D. Gao et al., "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," *arXiv preprint arXiv:2308.15363*, 2023. doi: 10.48550/arXiv.2308.15363

[8] W. Mao, R. Wang, J. Guo, J. Zeng, C. Gao, P. Han, and C. Liu, "Enhancing Text-to-SQL Parsing through Question Rewriting and Execution-Guided Refinement," in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, Aug. 2024, pp. 2009–2024. doi: 10.18653/v1/2024.findings-acl.120

[9] Z. Wang, R. Zhang, Z. Nie, and J. Kim, "ToolSQL: A Tool-Assisted Agent for SQL Verification and Refinement," in *Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining V.2 (KDD '25)*, Toronto, Canada, 2025, pp. 3102–3110. doi: 10.1145/3711896.3737159

[10] Z. Hong, Z. Yuan, H. Chen, Q. Zhang, F. Huang, and X. Huang, "Knowledge-to-SQL: Enhancing SQL Generation with Data Expert LLM," in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, Aug. 2024, pp. 10997–11008. doi: 10.18653/v1/2024.findings-acl.653

[11] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, "Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 12, pp. 7328–7345, 2025. doi: 10.1109/TKDE.2025.3609486

Appendix

The figures referenced in the main text are listed below for reference:

- Figure 1: TrustSQL Overall Architecture (`TrustSQL_Overall_Architecture.png`)
- Figure 2: Overall Accuracy — Baseline vs. Full Pipeline (`fig_overall_accuracy.png`)
- Figure 3: Accuracy by Difficulty Level (`fig_by_difficulty.png`)
- Figure 4: Ablation Study (`fig_ablation.png`)
- Figure 5: Robustness under Question Perturbation (`fig_robustness.png`)
- Figure 6: Error Type Distribution (`fig_error_distribution.png`)
- Figure 7: Latency Comparison (`fig_latency.png`)

```
============================================================
  TrustSQL — Interactive Demo
============================================================
Available databases (11):
   1. california_schools
   2. card_games
   3. codebase_community
   4. debit_card_specializing
   5. european_football_2
   6. financial
   7. formula_1
   8. student_club
   9. superhero
  10. thrombosis_prediction
  11. toxicology
============================================================
Type 'quit' to exit.

Question: What is the phone number of the school that has the highest average sc
ore in Math?
Database ID: california_schools
Evidence (optional, press Enter to skip):

=== Step 1: Filtered Schema ===
  satscores: ['cds', 'AvgScrMath']
  schools: ['CDSCode', 'Phone']

=== Step 2: CoT Reasoning ===
1. A single value is needed (the phone number of the school).
2. The tables likely involved are the schools and satscores tables.
3. Yes, a JOIN between tables is needed to connect school information with their
 corresponding scores.
4. Yes, there is a filtering condition to identify the school with the highest a
verage score in Math.
5. Yes, aggregation is needed to calculate the average score for each school.
6. Yes, sorting or row-limiting is needed to determine the school with the highe
st average score.

=== Step 3: Generated SQL ===
SELECT s.`Phone`
FROM schools s
JOIN satscores sc ON s.`CDSCode` = sc.`cds`
ORDER BY sc.`AvgScrMath` DESC
LIMIT 1;

=== Step 4: Verification ===
[VALID]

=== Step 5: Execution Result ===
  Rows: [('(408) 366-7700',)]

============================================================
Final SQL:  SELECT s.`Phone`
FROM schools s
JOIN satscores sc ON s.`CDSCode` = sc.`cds`
ORDER BY sc.`AvgScrMath` DESC
LIMIT 1;
Result:     [('(408) 366-7700',)]
============================================================
```

Figure A1: Interactive Demo Screenshot — TrustSQL five-step pipeline output.